Mars 2026

Disclaimer: This is an automated translation and may contain minor inaccuracies.

Technical User Workflow Synopsis

Design: Roland (Roland(Ypsos)) | Proofreading: Cricri

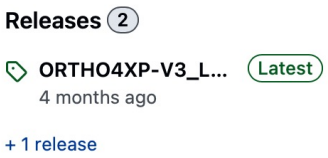
PART 1: (What the User Sees on Screen)

What the user concretely handles on the screen and how your scripts react behind the scenes:

ORTHO4XP-V3 MAC PRE-INSTALL

⚠️ Mac Only

On GITHUB, click on Releases at the bottom right.



Download FIRST: This ZIP contains

the `Lanceur_Installation_Prerequis.app` pre-cleaned to prevent Gatekeeper blocks.

Procedure:

🍎 Mac

- Download the main archive ORTHO4XP_V3 and unzip it.
- Download this ZIP and extract Lanceur_Installation_Prerequis.app directly into the unzipped ORTHO4XP_V3 folder.
- Place the ORTHO4XP_V3 folder into your Applications folder.
- Double-click on Lanceur_Installation_Prerequis.app.

Windows

- Download the main archive ORTHO4XP_V3 and unzip it.
- Double-click on LANCEUR_INSTALL_WINDOWS.bat.

- **Linux**

- Download the main archive ORTHO4XP_V3 and unzip it.
- Double-click on LANCEUR_INSTALL_LINUX.sh.

Python Modules Installation Launcher

Click on **Install Modules**.

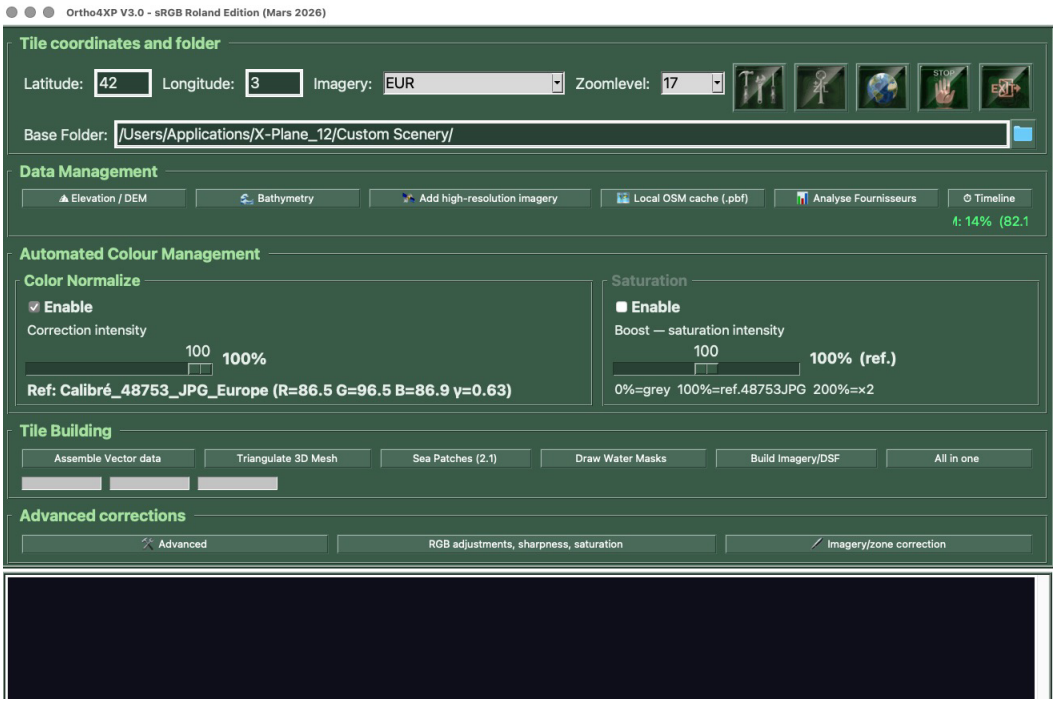
Select according to your system :1) Install modules, 2) Create Launcher

PRACTICAL USER GUIDE: Quick Start

A. Launching and Tile Preparation

➢ Launch Ortho4XP via the **launcher** (no terminal needed; for more details, see the step-by-step guide).

➢ In the main window: choose the **tile** by entering Latitude and Longitude coordinates.
➢ Select the imagery **provider** and the **Zoom Level**.
➢ Select the path to the Root folder or the folder where the tile will be created, for example: /Users/yourname/Applications/X-Plane_12/Custom Scenery/
➢ Click on the **TOOLS** icon.
➢ Under custom_dem: select Viewfinderpanoramas ....
➢ Under custom_overlay_src, navigate to the Global Scenery folder in X-Plane 12, for example: /Users/yourname/Applications/X-Plane_12/Global Scenery/X-Plane 12 Global Scenery
➢ Click the 4 buttons: **"Write tile cfg"**, **"Write app cfg"**, **"Apply"**, **"Quit"**.
➢ Click the **"All in 1"** button.



The **Installation and Quick Start** section is now complete.

Introduction to Additional Tools (V3 Version)

We will now explore the additional tools in Version 3.

Important Note: To discover all specific details, please refer to the Step-by-Step Tutorial document for each module.

A. Altimetry / Custom DEM

- In Advanced, open the Altimetry module.
- Specify the source DEM files.
-
- A single click launches the complete procedure (reprojection, cropping, merging, writing custom_dem).

What used to require **41 steps in QGIS** is now done automatically.

B. Relief and Coastline Sculpting (Bathymetry & Sea_Texture)

- **User Action:** The user sequences the generation steps for the 3D mesh, relief, and water bodies.
- Behind the Scenes: O4_Bathymetry carves out the seabed, and O4_Sea_Texture applies transparency and transition tints along the coastline for a realistic look in X-Plane.
- Complementary Modules: Altimetry module (one-click), O4_Bathymetry.py, and O4_Sea_Texture.py.

C. Creating a Custom Extent (If Needed)

- Click the "🛠️ Advanced" button (the single gateway to advanced tools).
- In the Advanced window, open the Extent Generator.
- Choose the mode:
- By tile (simple): enter the tile number $\rightarrow$ check the proposed zones $\rightarrow$ Click Create.
- By name (expert): enter the zone name or check "I already have an `.osm.bz2` file (JOSM trace)" then select the file.
- (Recommended) Open "⚙ Advanced Settings" to configure:
- Fineness (`pixel_size`): e.g., `10`
- Margin (`buffer`): e.g., `0` (or negative/positive depending on border offset)
- Blur (`blur`): e.g., `500`
- Click Create.
- $\rightarrow$ Behind the scenes: Launch of `O4_Mask_Utils.py`, generation of the `.ext` + `.png` + `.osm.bz2`trio, hot-reloading if possible.

D. Building the Tile (Classic Sequence)

In the main window, run the steps in the usual order:

- Orthophoto downloading
- (Optional) Color Check / Color Normalize
- Mesh / relief generation
- Masks and water (Sea Texture)
- DDS conversion and DSF finalization
- Follow the log console and progress bar (Orchestrator Pipeline).
- $\rightarrow$ Each step is named, timed, and secured (atomic transaction).

E. Correcting a Defective Image

- Open the Imagery Correction tool (via Advanced or dedicated button).
- View the textures and select the problematic tile.
- Send it to GIMP (or another editor).
- After editing, validate in Ortho4XP.
- $\rightarrow$ DDS compilation resumes only on the corrected area.

F. Geographical Editing (JOSM)Always via "🛠 Advanced".

- Choose the action (provider extent, leveling, airport, OSM layer...).
- JOSM opens automatically with the correct file.
- After editing and saving, return to Ortho4XP and validate.
- $\rightarrow$ Automatic backup + restoration capability.

G. Parameterization and Delimitation (Extent / Combo)

- User Action: The user selects their zone on the interactive map of the interface and adjusts geographical extents or source combinations.
- Behind the Scenes: The `extent` module mathematically frames the area, and the `combo` module prepares the overlay of chosen imagery layers, all memorized by `O4_Config.py`.
- Modules Involved: `O4_Extent_Generator.py` and `combo` / `lay` modules prepare the overlay. Configuration is memorized by O4_Config_Utils.py.
- Important Points:
- Two modes in the Extent Generator:
- By tile (simple): To quickly create extents from a tile's admin zones.
- By name (expert): To enter a name or use a pre-prepared file.
- Fine parameters (`pixel`, `buffer`, `blur`) are located behind "⚙ Advanced Settings".
- If the user already has an `.osm.bz2` (JOSM trace), they switch to expert mode, check "I already have an `.osm.bz2` file", and select their file.

H. Downloading and Quality Control (`color_check`)

- User Action: The user starts downloading images. The process runs safely in the background without blocking their connection, thanks to anti-ban security measures.
- Behind the Scenes: `O4_Custom_URL.py` regulates requests, then the `color_check` module scans downloaded tiles to assign a quality score and detect potential clouds or major defects.
- Modules Involved: `O4_HTTP_Client.py` manages requests, followed by `O4_Color_Check.py` + `O4_Provider_Score.py` which rate tile quality.

I. Color Harmonization (`color_normalise` & `color_appli`)

- User Action: The user enables the color-matching option to ensure landscapes are free from color aberrations from one photo to the next.
- Behind the Scenes: `color_normalise` computes the required exposure/sRGB corrections, and `color_appli` applies these modifications directly to source images.
- Modules
Involved: `O4_Color_Normalize.py` calculates, `O4_Color_Apply.py` applies.
Original sources are left unmodified.

J. Retouches and External Bridges (GIMP, QGIS, JOSM, Patches)

- User Action: Facing a persistent image defect, the user clicks the tile thumbnail to send it directly to their external editing software or GIS tool, then validates the correction.
- Practical Action:
- Open the Imagery Correction tool (via Advanced or dedicated button).
- View textures and select the problematic tile.
- Send it to GIMP (or another editor).
- After editing, validate in Ortho4XP.
- Behind the Scenes: The system pauses DDS conversion for safety. The O4_Eraser module (or Patchesmanagement) integrates the validated modification via the GIMP / QGIS / JOSM gateway before authorizing final compilation.
- Modules Involved: O4_Correction_Utils.py + Advanced modules re-integrate the correction.
- **Targeted Finalization:** DDS compilation resumes solely on the corrected area.
-

-

K. Finalization

- Memory monitoring
- Event Bus
- Orchestrator Pipeline
- V2 Compatibility
- No terminal required
-

User Points of Vigilance

- The most powerful options (.osm.bz2, pixel/buffer/blur, JOSM, Altimetry) are grouped behind the "🛠️ Advanced" button.
- Fine extent settings are collapsed: always open "⚙ Advanced Settings" if the result is unsatisfactory.
- When in doubt, check the log console: every action by Roland(Ypsos) clearly writes what it is doing there.
- No terminal is needed once installation is complete via the launchers.
-

PART 2: Detailed Technical Architecture and Specificities by File

File / Module Name	Ypsos Specificity / Technical Role in the Architecture
O4_Color_Check.py	**Ypsos Specificity (Image analysis)**: Mathematically analyses every downloaded tile (noise, excessive JPEG compression, clouds, colour drift, seam risk) to generate a quality score and detect anomalies before compilation. Completely new module, absent from version 1.40.
O4_Color_Normalize.py	**Ypsos Specificity (Colour harmonisation)**: Applies neutral sRGB correction algorithms, adapted to the zoom level, to smooth differences in hue and contrast between adjacent tiles or heterogeneous sources. Validated for X-Plane 12 HDR. Did not exist in previous versions.
O4_Color_Apply.py	**Ypsos Specificity (Profile application)**: Applies the calculated correction matrices to the final textures at the right point in the processing chain, without ever modifying the original sources.
O4_Provider_Score.py + O4_Score_Logger.py	**Ypsos Specificity (Provider scoring)**: Automatically rates every image (noise, compression, clouds, drift, seam risk) and designates the best provider. New in V3.
O4_Correction_Utils.py	**Ypsos Specificity (Imagery correction / Correction mosaic)**: Visualises DDS textures, isolates “no-data” or corrupted tiles, allows selection, quarantine, sending to an external editor and targeted regeneration. Replaces and extends the Eraser/Patches concept.
O4_Extent_Generator.py	**Ypsos Specificity (Geometric extent management)**: Calculates, cuts and automatically generates `.ext` files + OSM archive (By tile / By name modes, `.osm.bz2` support, pixel/buffer/blur settings). Automates what was manual in 1.40. *Contributions: Jasum, Len0y, domisilasol (Dominique).*
O4_comb_generator.py + O4_lay_generator.py	**Ypsos Specificity (Layer fusion / .lay generator)**: Dynamically combines multiple imagery sources or superimposes multiple masks. Automatic generation of `.lay` files and support for ultra high-definition imagery (PCRS_IGN, IGN Ortho). *Contributions: Jasum, Len0y, domisilasol (Dominique).*
O4_Sea_Texture.py	**Ypsos Specificity (Coastline processing and XP12)**: Creates and manages photorealistic sea patches matching the orthophotos, transparent transition masks and BC3 textures with alpha channel. Permanently eliminates blue squares and transparent triangles. Compatible with all providers and all zoom levels. Completely new module.
O4_Bathymetrie_Utils.py	**Ypsos Specificity (Bathymetry utilities)**: Complementary tools for underwater modelling. New module.
O4_Bathymetry.py	**Based on the original engine (Oscar Pilote / Shred86)** — adapted.

File / Module Name	Ypsos Specificity / Technical Role in the Architecture
O4_HTTP_Client.py + O4_Provider_Abstraction.py	**Ypsos Specificity (Download security and robustness)**: Manages HTTP headers, multi-threading, automatic server rotation, recovery after failure, white-tile handling and dynamic anti-ban timing (especially for IGN).
O4_Config_Utils.py	**Based on the original engine (Oscar Pilote / Shred86)**: Centralises reading, writing and validation of `.cfg` parameters. Modernised and strengthened version. *Contributions : Jasum, Len0y, domisilasol (Dominique).*
O4_GUI_Utils.py + O4_UI_Utils.py	**Based on the original engine (Oscar Pilote / Shred86)**: Main interface, interactive map, logs and settings. Lightly adapted to integrate the new modules in a non-blocking way. Original files.
O4_UI_Dialogs.py	**Ypsos Specificity**: Interface dialogs. New module.
O4_Theme_Manager.py	**Ypsos Specificity (Theme manager)**: 5 predefined themes + full customisation of interface colours.
O4_EventBus.py	**Ypsos Specificity (Event-driven architecture)**: Decoupled communication between modules. Modern architecture introduced in V3.
Pipeline Orchestrator	**Ypsos Specificity**: Every build step (download → conversion → DSF) is named, timed and publishes its status in real time. In case of failure, no file is corrupted.
O4_Build_Transaction.py	**Ypsos Specificity (Atomic transactions)**: Guarantees tile integrity during the build.
O4_Memory_Manager.py	**Ypsos Specificity (Memory monitoring)**: Real-time RAM monitoring + automatic cache cleaning before saturation.
O4_Dependency.py + Intelligent Cache	**Ypsos Specificity**: Remembers the parameters of each tile. If the settings are identical, the rebuild is automatically skipped.
GPU Backend	**Ypsos Specificity**: Automatic CUDA acceleration with silent fallback to CPU.
O4_Altimetrie_Utils.py + associated Relief modules	**Ypsos Specificity (One-click altimetry)**: Turns the 41 manual QGIS steps into a single click (reprojection, clipping, merging, writing `custom_dem`). Folder structure is created automatically.
O4_Menu_Avance.py + O4_Avance_Utils.py	**Ypsos Specificity (Advanced JOSM/QGIS module)**: Automatic launch of JOSM, secure geographic editing (provider extents, levelling, airports, OSM layers), backup/restore, anti-upload to OpenStreetMap.
O4_Airport_Utils.py	**Based on the original engine (Oscar Pilote / Shred86)** — adapted (ICAO code reading and levelling patches).
O4_XP12_Materials.py	**Ypsos Specificity**: Management of materials and native X-Plane 12 specificities. Calculator (Altimetry)** | One-click module (reprojection, fusion, automatic writing)
O4_Lang.py + O4_Lang_*.py	**Ypsos Specificity**: Multilingual system.

File / Module Name	Ypsos Specificity / Technical Role in the Architecture
External Gateways (GIMP, QGIS, JOSM)	**Ypsos Specificity (Third-party tool integration)**: Link and lock modules that allow instant transfer of a texture or vector to GIMP, QGIS or JOSM, then wait for human validation before resuming DDS compilation.
Launchers and Automatic Installation	**Ypsos Specificity (Zero Terminal)**: `Ortho4XP_Launcher.py`, `INSTALL_PREREQUIS.py`, `LANCEUR_INSTALL_LINUX.sh`, `LANCEUR_INSTALL_WINDOWS.bat`, `Lanceur_Installation_Prerequis.app`, native launcher creation scripts (`.app` / `.vbs` / `.desktop`). One-click graphical installation, automatic detection and installation of Python 3.12, creation of an isolated `venv` environment.
Original engine (Oscar Pilote / Shred86 1.40)	Kept intact: mesh calculation, DSF generation, basic tile logic, Triangle4XP, Mask_Utils, Imagery_Utils, etc. No core file is modified. All Ypsos modules are autonomous: if one is missing or fails, Ortho4XP still starts and works.